

Desenvolvedor(a) em IA para Análise Preditiva [T2]

Mini-Projeto Avaliativo - Módulo 1 - Semana 08

SUMÁRIO

1. CONTEXTUALIZAÇÃO	1
2. DESAFIO	2
3. RESULTADOS ESPERADOS (O QUE VOCÊ DEVE ENTREGAR)	3
4. REQUISITOS DAS TAREFAS	4
4.1. ORGANIZAÇÃO DO PROJETO	4
4.2. REQUISITOS FUNCIONAIS	4
Passo 1: Bloco de importação de bibliotecas — cole e use-as no topo do seu notebook dataview.ipynb	4
RF01 – Criar ou Carregar o Dataset de Vendas	4
RF02 – Inspecionar e Descrever os Dados	6
RF03 – Limpar e Tratar os Dados	7
RF04 – Detectar e Tratar Outliers (versões v1 e v2)	8
RF05 – Criar Colunas Derivadas com Transformações	9
RF06 – Calcular Métricas Agregadas (groupby)	10
RF07 – Segmentar Clientes por Nível de Gasto	12
RF08 – Calcular Estatísticas com NumPy	13
RF09 – Criar Visualizações com Matplotlib e Seaborn	14
RF10 – Organizar o Código em Funções Reutilizáveis	16
RF11 – Ler e Escrever Arquivos (CSV e JSON)	17
RF12 – Consolidar a Análise e Salvar o Dataset Final	17
4.3. REQUISITOS TÉCNICOS	18
4.4. VERSIONAMENTO COM GITHUB	20
4.5. GRAVAÇÃO DE VÍDEO	20
5. CRITÉRIOS DE AVALIAÇÃO	20
6. CHECKLIST FINAL DE ENTREGA	22
7. MODELO DE README.md (SUGESTÃO)	23

1. CONTEXTUALIZAÇÃO

O mercado de Inteligência Artificial e Ciência de Dados exige que você demonstre raciocínio lógico, organização, capacidade de resolver problemas com dados reais e boa comunicação técnica. Mesmo antes de construir modelos preditivos avançados com redes neurais ou

aprendizado de máquina, já é possível desenvolver um projeto completo e útil para seu portfólio usando Python, Pandas, NumPy e visualizações.

Neste miniprojeto, você é um Analista de Dados Júnior que recebeu um dataset de vendas com problemas e precisa entregá-lo limpo, analisado e visualizado para identificar padrões e gerar respostas a possíveis questões que possam surgir em uma reunião da diretoria do seu setor na empresa.

Este projeto engloba as seguintes atividades de um analista de dados

- ✓ Coletar e carregar os dados
- ✓ Inspecionar e entender o dataset
- ✓ Limpar dados sujos (nulos, erros, strings mal formatadas)
- ✓ Criar novas colunas derivadas
- ✓ Resumir e agregar com groupby
- ✓ Calcular estatísticas com NumPy
- ✓ Visualizar com gráficos
- ✓ Exportar relatórios em CSV e JSON

2. DESAFIO

Você acaba de ser contratado como Analista de Dados Júnior em uma empresa de varejo que possui um histórico de vendas em formato CSV e precisa urgentemente de um relatório analítico para a reunião trimestral da diretoria. O time de negócios quer entender:

- Como as vendas se comportam ao longo do tempo (por mês, por trimestre);
- Quais produtos e categorias geram mais receita;
- Quais regiões têm melhor desempenho;
- Quais clientes são mais valiosos (segmentação simples);

Você deverá desenvolver um miniprojeto em Python chamado:

DataView: Exploração e Análise de Dados de Vendas

O objetivo do projeto é criar um fluxo completo de análise de dados que leia, limpe, transforme, analise e visualize um dataset de vendas, gerando insights relevantes e um relatório exportável.

A aplicação deverá:

- Ler e inspecionar os dados brutos de um arquivo CSV;
- Limpar e tratar dados inconsistentes (valores nulos, formatos de data errados, espaços extras em strings);
- Criar novas colunas derivadas (receita total por linha, mês, trimestre, ano);
- Calcular métricas agregadas por produto, categoria e região;
- Segmentar clientes por nível de gasto (Bronze, Prata, Ouro);

- Gerar visualizações claras e informativas;
- Exportar um relatório resumido em CSV e as estatísticas gerais em JSON;
- Organizar todo o fluxo de análise de dados em funções reutilizáveis para futuras implementações (essa ação ajudaria você a otimizar a análise do mesmo dataset em reuniões futuras com a diretoria).

Maneira de Execução

O projeto poderá ser executado de uma destas formas:

- **Google Colab (recomendado para a disciplina):** basta fazer upload do arquivo .py ou .ipynb e rodar as células;
- **VS Code** com extensão Python e o arquivo .py rodando localmente via terminal;
- **Terminal/Prompt de Comando:** python dataview.py (requer Python e as bibliotecas instaladas via pip).

O Google Colab já possui as bibliotecas Pandas, NumPy, Matplotlib e Seaborn instaladas. Para execução local, será necessário instalar as dependências com o comando *pip install pandas numpy matplotlib seaborn*.

3. RESULTADOS ESPERADOS (O QUE VOCÊ DEVE ENTREGAR)

Ao final do miniprojeto, o estudante ou squad (pequeno grupo de até 3 pessoas) deverá entregar:

1. Arquivo Python principal: notebook dataview.ipynb (ou dataview.py)
2. Arquivo vendas.csv bruto e as suas versões de tratamento com os dados limpos
3. README.md explicando o projeto (modelo na Seção 7)
4. Repositório público no GitHub
5. Vídeo de até 3 minutos demonstrando o funcionamento e explicando as decisões
6. Links do seu repositório no GitHub e do vídeo enviados no AVA

Entrega no AVA

A entrega deverá ser submetida na tarefa do AVA: "Módulo 01 – Mini-Projeto Avaliativo" na semana 08.

Prazo: **24/06/2026 até às 22h**

Peso do mini-projeto na nota: Avaliação M1.1 – 25% da nota do módulo.

Links obrigatórios no AVA

O estudante deverá enviar no AVA:

- Link do repositório público no GitHub
- Link do vídeo (carregar o vídeo no seu Google Drive com acesso público ou carregar o vídeo no seu canal do YouTube como vídeo não listado)

4. REQUISITOS DAS TAREFAS

4.1. ORGANIZAÇÃO DO PROJETO

Organize seu projeto com estrutura similar à seguinte:

```
projeto/
├── data/      # Todos os dados do projeto, separados por estágio do pipeline
│   ├── raw/      # Salve seus dados brutos, originais e imutáveis
│   ├── processed/ # Salve os dados já limpos e transformados
│   │   ├── v1_com_outliers/ # Salve os dados com limpeza geral, mas com os outliers mantidos
│   │   └── v2_outliers_tratado/ # Salve os dados com limpeza geral v1 + tratamento de outliers
│   └── final/     # Dados que serão usados futuramente em um treinamento de modelo de IA
├── notebooks/   # Notebooks para análise exploratória e experimentos
├── outputs/      # Salvar resultados finais, relatórios, e visualizações
│   ├── metricas_por_mes.csv # Resumo de métricas mensais extraído dos dados
│   ├── segmentacao_clientes.csv # Resultado da segmentação de clientes feita no notebook
│   ├── estatisticas_gerais.json # Resumo estatístico do dataframe
│   └── graficos/      # Visualizações exportadas para análise e apresentações
└── README.md     # Documentação do projeto, estrutura de pastas e justificativas de decisão
```

Não é necessário criar todas as pastas. O foco é demonstrar domínio dos conceitos de Python e de análise de dados estudados.

4.2. REQUISITOS FUNCIONAIS

Acompanhe conosco o passo a passo de cada etapa:

Passo 1: Bloco de importação de bibliotecas — cole e use-as no topo do seu notebook `dataview.ipynb`

Todas as importações devem estar no início do arquivo como parte das boas práticas de desenvolvimento do seu código:

```
import pandas as pd
import numpy as np
import random
import os
import json
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime, timedelta
```

RF01 – Criar ou Carregar o Dataset de Vendas

O que é esperado

O projeto deve usar um dataset de vendas com ao menos as colunas:

id | data | cliente | produto | categoria | regioao | quantidade | preco

O dataset deve conter dados intencionalmente sujos: valores None, datas inválidas e espaços extras em strings — para você praticar a limpeza na RF03.

O projeto deverá incluir um dataset de vendas. O estudante pode **gerar o dataset sinteticamente** no próprio código (recomendado para facilitar a reprodução) ou utilizar um arquivo CSV real.

Opção A – Gerar o dataset no código (recomendado):

```
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
import random

def gerar_dataset_vendas(n_registros=150, seed=42):
    """Gera um dataset sintético de vendas com dados propositalmente sujos,
    incluindo valores nulos, strings sujas, datas inválidas e outliers."""
    random.seed(seed)
    np.random.seed(seed)

    produtos = ['Notebook', 'Smartphone', 'Tablet', 'Monitor', 'Teclado',
                'Mouse']
    precos = { 'Notebook': 3500, 'Smartphone': 2200, 'Tablet': 1800,
               'Monitor': 1200, 'Teclado': 250, 'Mouse': 120 }
    categorias = { "Notebook": "Computadores", "Smartphone": "Celulares",
                   "Tablet": "Celulares", "Monitor": "Computadores", "Teclado": "Periféricos",
                   "Mouse": "Periféricos" }
    regioes = ["Sudeste", "Sul", "Nordeste", "Centro-Oeste", "Norte"]
    clientes = [f"Cliente_{i:03d}" for i in range(1, 31)]

    data_inicio = datetime(2024, 1, 1)
    dados = []

    for i in range(n_registros):
        produto = random.choice(produtos)
```

```
quantidade = random.randint(1, 10)
preco = precos[produto]
data = data_inicio + timedelta(days=random.randint(0, 364))

# Inserindo dados intencionalmente sujos para limpeza
if random.random() < 0.05:
    quantidade = None          # valor nulo
if random.random() < 0.04:
    preco = None               # valor nulo
if random.random() < 0.03:
    produto = " " + produto   # espaço extra (string suja)

data_str = data.strftime("%Y-%m-%d") if random.random() > 0.02 else "DATA
INVALIDA"

dados.append({
    "id_venda": i + 1,
    "data_venda": data_str,
    "cliente": random.choice(clientes),
    "produto": produto,
    "categoria": categorias.get(produto.strip(), "Outros"),
    "regiao": random.choice(regioes),
    "quantidade": quantidade,
    "preco_unitario": preco
})

return pd.DataFrame(dados)

# Gerar e salvar
df_bruto = gerar_dataset_vendas()
os.makedirs('data/raw', exist_ok=True) # Adicionado para garantir que o
diretório exista
df_bruto.to_csv("data/raw/vendas.csv", index=False)
print(f"Dataset gerado com {len(df_bruto)} registros.")
print(df_bruto.head())
```

Alternativa: usar CSV externo

Você pode baixar um dataset real externo (ex: Kaggle, dados.gov.br) e adaptá-lo,

desde que contenha ao menos as colunas: data, produto, categoria, quantidade, preço e região.

RF02 – Inspecionar e Descrever os Dados

O projeto deverá inspecionar o dataset carregado e exibir informações básicas. Exemplo de resultado esperado no console:

```
=== INSPEÇÃO INICIAL DO DATASET ===
Shape: (150, 8)
Colunas: ['id_venda', 'data_venda', 'cliente', 'produto', 'categoria',
'regiao', 'quantidade', 'preco_unitario']
Tipos de dados:
id_venda          int64
data_venda        object
cliente           object
...
Valores nulos por coluna:
quantidade        10
preco_unitario     8
...
Primeiros registros:
id_venda  data_venda      cliente  produto ...
```

Exemplo de função:

```
def inspecionar_dados(df):
    """Exibe informações básicas do DataFrame."""
    print("\n=== INSPEÇÃO INICIAL DO DATASET ===")
    print(f"Shape: {df.shape}")
    print(f"\nColunas: {list(df.columns)}")
    print(f"\nTipos de dados:\n{df.dtypes}")
    print(f"\nValores nulos por coluna:\n{df.isnull().sum()}")
    print(f"\nPrimeiros registros:\n{df.head()}")
    print(f"\nEstatísticas descritivas:\n{df.describe()}")
    return df.describe(include="all")
```

RF03 – Limpar e Tratar os Dados

O projeto deverá realizar a limpeza dos dados. O estudante deve tratar ao menos os seguintes problemas:

- Remover ou imputar linhas com valores nulos nas colunas críticas (quantidade, preco_unitario);

- Remover linhas com datas inválidas;
- Converter a coluna de data para o tipo datetime;
- Remover espaços extras em colunas de texto com `.str.strip()`;
- Registrar no console quantos registros foram removidos.

Exemplo:

```
import re
import os
# LIMPEZA COM EXPRESSÕES REGULARES (módulo re)
# re.sub(padrao, substituto, string) substitui todas as ocorrências
# do padrão pela string substituta.
# r"\s+" é um padrão regex que significa "um ou mais espaços em branco"
# (incluindo espaços, tabs e quebras de linha).
# -----
def limpar_strings_regex(df, colunas):
    """ Usa expressões regulares para normalizar colunas de texto: -
    Colapsa múltiplos espaços internos em um único espaço (re.sub) - Remove
    espaços nas pontas da string (.strip()) - Preserva células nulas sem lançar
    erro (pd.notna) """
    df = df.copy() # Não modifica o DataFrame original
    for col in colunas:
        df[col] = df[col].apply(
            # pd.notna(s): verifica se o valor NÃO é nulo antes de processar
            # re.sub(r"\s+", " ", str(s)): substitui sequências de espaços por
            um único espaço
            # .strip(): remove espaços residuais nas pontas
            lambda s: re.sub(r"\s+", " ", str(s)).strip() if pd.notna(s) else
            s)
    return df

def limpar_dados(df):
    """ Limpa o DataFrame de vendas em quatro etapas: 1. Normaliza strings
    com regex (espaços extras) 2. Converte datas e remove registros com datas
    inválidas 3. Remove linhas com valores nulos em colunas obrigatórias 4.
    Garante os tipos numéricos corretos Retorna: (df_limpo, relatorio) - o
    relatório documenta o impacto de cada etapa. """
    df = df.copy()
    n_inicial = len(df)
    relatorio = {}
    # --- Etapa 1: limpeza de strings com regex ---
```

```
# select_dtypes("object") seleciona apenas colunas de texto
colunas_texto = df.select_dtypes(include="object").columns
df = limpar_strings_regex(df, colunas_texto)
# --- Etapa 2: conversão de datas ---
# errors="coerce" transforma valores inválidos (ex: "31/02/2024") em
NaT em vez de lançar um erro - depois removemos essas linhas com dropna

df["data_venda"] = pd.to_datetime(df["data_venda"], errors="coerce")

relatorio["datas_invalidas_removidas"] =
int(df["data_venda"].isnull().sum())

df = df.dropna(subset=["data_venda"])

# --- Etapa 3: remoção de nulos em colunas obrigatórias ---

# Uma linha sem quantidade ou preço não pode contribuir para nenhuma
métrica; # por isso optamos por remover (em vez de imputar) esses
registros.
n_antes = len(df)
df = df.dropna(subset=["quantidade", "preco_unitario"])
relatorio["linhas_nulas_removidas"] = n_antes - len(df)

# --- Etapa 4: garantia de tipos numéricos --- # Após o dropna, os
valores existentes podem ainda estar como float/object; # forçamos int
para quantidade e float para preço.
df["quantidade"] = df["quantidade"].astype(int)
df["preco_unitario"] = df["preco_unitario"].astype(float)

# --- Relatório final ---
relatorio["registros_iniciais"] = n_inicial
relatorio["registros_finais"] = len(df)
relatorio["registros_removidos_total"] = n_inicial - len(df) # soma de
todas as remoções
print("=== RELATORIO DE LIMPEZA ===")
for k, v in relatorio.items():
    print(f" {k}: {v}")

return df, relatorio
```

```
# EXECUÇÃO: limpar o dataset bruto e salvar como versão v1 (com outliers)
# Nesta versão os outliers são MANTIDOS – eles serão tratados na RF04.
df_v1, relatorio = limpar_dados(df_bruto)
os.makedirs("data/processed/v1_com_outliers", exist_ok=True) # cria o diretório se não existir
df_v1.to_csv("data/processed/v1_com_outliers/vendas_v1.csv", index=False)
print("\nv1 salva em data/processed/v1_com_outliers/")
df_v1.head()
```

RF04 – Detectar e Tratar Outliers (versões v1 e v2)

Após a limpeza geral (RF03), o estudante deve trabalhar os outliers das colunas numéricas e gerar duas versões dos dados, salvas em data/processed/:

- v1_com_outliers/ — os dados limpos da RF03, com os outliers mantidos;
- v2_outliers_tratado/ — a mesma base da v1, agora com os outliers tratados (removidos ou substituídos).

Espera-se que o estudante: detecte os outliers com um método estatístico (IQR ou z-score); gere as limpezas de dados v1 e v2; e registre no notebook quantos outliers foram encontrados. O ponto central é a reutilização: a mesma função de limpeza alimenta ambas as versões.

Exemplo de detecção e tratamento por IQR:

```
def tratar_outliers(df, colunas, fator=1.5, metodo='remover'):
    """
    Trata outliers de colunas numéricas usando o Intervalo Interquartil (IQR).

    Parâmetros:
        colunas : lista de colunas numéricas a verificar
        fator    : multiplicador do IQR para definir os limites (padrão=1.5)
        metodo   : 'remover' exclui as linhas com outliers;
                   'limitar' aplica winsorização (substitui pelo limite)

    Retorna o DataFrame tratado sem modificar o original (usa .copy()).
    """
    df = df.copy() # garante que df_v1 não seja alterado fora da função
    for col in colunas:
        # Q1 = 25% dos dados estão abaixo desse valor
        # Q3 = 75% dos dados estão abaixo desse valor
```

```
q1 = df[col].quantile(0.25)
q3 = df[col].quantile(0.75)
iqr = q3 - q1 # largura do intervalo central (50% dos dados)

lim_inf = q1 - fator * iqr # abaixo disso = outlier inferior
lim_sup = q3 + fator * iqr # acima disso = outlier superior

# Conta quantas linhas estão fora dos limites (| = OR lógico)
n_out = ((df[col] < lim_inf) | (df[col] > lim_sup)).sum()
print(f' {col}: {n_out} outliers detectados '
      f'(lim_inf={lim_inf:.2f}, lim_sup={lim_sup:.2f})')

if metodo == 'remove':
    # Mantém apenas as linhas dentro dos limites
    df = df[(df[col] >= lim_inf) & (df[col] <= lim_sup)]
else:
    # Winsorização: em vez de remover, "apara" os valores extremos
    # pelo limite – nenhuma linha é perdida, só os valores mudam.
    df[col] = df[col].clip(lower=lim_inf, upper=lim_sup)

return df

# GERAÇÃO DA v2: mesma base da v1, com outliers tratados

# Problema: 'receita_total' (quantidade × preco_unitario) ainda não existe
# em df_v1 – ela só será criada oficialmente no RF05. Mas queremos detectar
# outliers de receita aqui, antes de derivar colunas.

# Solução: criamos uma cópia temporária (df_v1_tmp) só para o cálculo
# de outliers, e depois descartamos a coluna. Assim o RF05 recria
# 'receita_total' de forma consistente para toda a análise posterior.

# Cópia temporária para calcular receita_total sem alterar df_v1
df_v1_tmp = df_v1.copy()
df_v1_tmp["receita_total"] = df_v1_tmp["quantidade"] *
df_v1_tmp["preco_unitario"]
```

```
# Detecta e remove outliers em quantidade e receita_total
# (preco_unitario não é tratado separadamente porque seu efeito
# já aparece em receita_total junto com quantidade)
df_v2 = tratar_outliers(
    df_v1_tmp,
    colunas=["quantidade", "receita_total"],
    metodo='remover'
)
# Remove a coluna temporária: ela será recriada no RF05 sobre df_v2 limpo
df_v2 = df_v2.drop(columns=["receita_total"])
# Comparativo de linhas para confirmar quantos outliers foram removidos
print(f"\nv1 = {len(df_v1)} linhas (com outliers)")
print(f"\nv2 = {len(df_v2)} linhas (outliers removidos)")
print(f"Diferença = {len(df_v1) - len(df_v2)} linhas removidas")

os.makedirs("data/processed/v2_outliers_tratado", exist_ok=True)
df_v2.to_csv("data/processed/v2_outliers_tratado/vendas_v2.csv", index=False)
print("\nv2 salva em data/processed/v2_outliers_tratado/")
```

No README, especifique qual versão (v1 ou v2) foi escolhida para data/final/.

RF05 – Criar Colunas Derivadas com Transformações

O projeto deverá criar novas colunas a partir dos dados existentes, enriquecendo o dataset para análise. Deve-se criar ao menos:

- receita_total: quantidade * preco_unitario
- mes: mês extraído da data (número ou nome)
- trimestre: trimestre do ano (Q1, Q2, Q3, Q4)
- ano: ano extraído da data
- faixa_receita_item: classificação da receita por item usando transformação condicional

Exemplo:

```
def criar_colunas_derivadas(df):
    """
    Cria colunas calculadas a partir do dataset limpo:
    - receita_total : valor total da linha de venda (quantidade × preço)
    - mes / trimestre / ano : componentes extraídos da data
    - faixa_receita_item : classificação do valor de cada venda (np.select)
```

Nota: receita_total foi calculada temporariamente no RF04 apenas para detecção de outliers e depois descartada. Aqui ela é criada de forma definitiva sobre df_v2 – já com os outliers tratados – para que todas as análises posteriores usem a mesma base consistente.

```
"""
df = df.copy()

# Receita por linha: grandeza central de todas as métricas do projeto
df["receita_total"] = df["quantidade"] * df["preco_unitario"]

# Componentes de data – o atributo .dt expõe propriedades de datetime
df["mes"] = df["data_venda"].dt.month
df["trimestre"] = df["data_venda"].dt.quarter.apply(lambda q: f"Q{q}")
df["ano"] = df["data_venda"].dt.year

# -----
# np.select: alternativa vetorizada ao if/elif/else para criar
# colunas categóricas. Recebe duas listas de mesmo tamanho:
#   condicoes : lista de máscaras booleanas (testadas em ordem)
#   valores   : rótulo retornado quando a condição correspondente é True
#   default   : valor usado quando nenhuma condição é satisfeita
# É equivalente a apply(lambda x: ...) mas muito mais eficiente em
# datasets grandes, pois opera sobre arrays sem usar loops Python.
# -----
condicoes = [
    df["receita_total"] < 500,
    (df["receita_total"] >= 500) & (df["receita_total"] < 5000),
    df["receita_total"] >= 5000,
]
rotulos = ["Baixo Valor", "Médio Valor", "Alto Valor"] # alinhado ao
enunciado
df["faixa_receita_item"] = np.select(condicoes, rotulos, default="N/D")

print("COLUNAS DERIVADAS CRIADAS")
print(df[["data_venda", "receita_total", "mes", "trimestre",
"faixa_receita_item"]].head())
```

```
return df
```

```
# A partir daqui, `df` é o DataFrame principal de toda a análise:
# dataset v2 (outliers tratados) + colunas derivadas
df = criar_colunas_derivadas(df_v2)
df[["data_venda", "receita_total", "mes", "trimestre",
    "faixa_receita_item"]].head()
```

RF06 – Calcular Métricas Agregadas (groupby)

O sistema deverá calcular e exibir métricas agrupadas por diferentes dimensões. Deve-se gerar ao menos:

- Receita total e quantidade vendida **por mês**;
- Receita total **por produto** (top 5);
- Receita total **por categoria**;
- Receita total **por região**.

Exemplo de resultado esperado no console:

```
=== RECEITA POR MÊS ===
   mes  receita_total  quantidade
1    1      45230.50         312
2    2      38910.00         289
...

=== TOP 5 PRODUTOS POR RECEITA ===
   produto  receita_total
1  Notebook      125430.00
2  Smartphone       98320.50
...
```

Exemplo de implementação:

```
def calcular_metricas(df):
    """
    Calcula e retorna um dicionário com métricas agregadas por
    quatro dimensões: mês, produto, categoria e região.
    Usa .groupby() + .agg() com nomeação explícita de colunas:
        nova_coluna=("coluna_origem", "função")
    Isso permite criar múltiplas agregações em uma única chamada
```

e nomear cada resultado diretamente, sem precisar renomear depois.

```
"""
```

```
metricas = {}
```

```
# --- Receita e volume por mês ---
```

```
# n_vendas conta quantas transações ocorreram em cada mês
```

```
metricas["por_mes"] = (
```

```
    df.groupby("mes")
```

```
    .agg(
```

```
        receita_total=("receita_total", "sum"),      # soma da receita no mês
```

```
        quantidade=("quantidade", "sum"),           # total de itens
```

```
vendidos
```

```
        n_vendas=("id_venda", "count"),             # número de transações
```

```
    )
```

```
    .reset_index()
```

```
    .sort_values("mes")
```

```
)
```

```
# --- Top 5 produtos por receita total ---
```

```
# .head(5) após sort_values garante o ranking dos maiores
```

```
metricas["top_produtos"] = (
```

```
    df.groupby("produto")["receita_total"]
```

```
    .sum()
```

```
    .sort_values(ascending=False)
```

```
    .head(5)
```

```
    .reset_index()
```

```
)
```

```
# --- Receita por categoria ---
```

```
metricas["por_categoria"] = (
```

```
    df.groupby("categoria")["receita_total"]
```

```
    .sum()
```

```
    .reset_index()
```

```
    .sort_values("receita_total", ascending=False)
```

```
)
```

```
# --- Receita e ticket médio por região ---
```

```
# media_ticket = receita média por transação individual em cada região;
```

```
# permite comparar regiões além do volume total (uma região pode ter
# menos vendas, mas vendas de maior valor)
metricas["por_regiao"] = (
    df.groupby("regiao")
      .agg(
        receita_total=("receita_total", "sum"),
        media_ticket=("receita_total", "mean"),
      )
      .reset_index()
      .sort_values("receita_total", ascending=False)
)

# Exibição resumida para conferência no notebook
for nome, tabela in metricas.items():
    print(f"\n=== {nome.upper().replace('_', ' ')} ===")
    print(tabela.to_string(index=False))
return metricas

metricas = calcular_metricas(df)
```

RF07 – Segmentar Clientes por Nível de Gasto

O sistema deverá agrupar os dados por cliente, calcular o total gasto de cada um e classificá-los em segmentos. Deve-se usar pelo menos uma função lambda e uma transformação condicional.

Critério sugerido de segmentação:

Gasto Total	Segmento
Abaixo de R\$ 5.000	Bronze
R\$ 5.000 a R\$ 15.000	Prata
Acima de R\$ 15.000	Ouro

Exemplo:

```
def segmentar_clientes(df):
    """
    Agrupa os dados por cliente, calcula o total gasto por cada um
    e classifica em Bronze / Prata / Ouro conforme os limites abaixo:
```

< R\$ 5.000 → Bronze
R\$ 5.000–R\$ 15.000 → Prata
> R\$ 15.000 → Ouro

Demonstra o uso de função lambda com condicional encadeado –
equivalente a um if/elif/else em uma única expressão.

"""

Soma a receita total de cada cliente em todas as suas compras

```
clientes_df = (  
    df.groupby("cliente")["receita_total"]  
        .sum()  
        .reset_index()  
)  
clientes_df.columns = ["cliente", "total_gasto"]
```

Lambda com ternário aninhado – como ler:

```
# "Se g > 15000 → 'Ouro'  
#   Senão, se g >= 5000 → 'Prata'  
#   Senão → 'Bronze'"  
#
```

Nota: g > 15000 significa que exatamente R\$ 15.000 cai em Prata,
consistente com o critério "acima de R\$ 15.000" para Ouro.

```
clientes_df["segmento"] = clientes_df["total_gasto"].apply(  
    lambda g: "Ouro" if g > 15000 else ("Prata" if g >= 5000 else "Bronze")  
)  
clientes_df = clientes_df.sort_values("total_gasto", ascending=False)  
print("=== SEGMENTAÇÃO DE CLIENTES (Top 10) ===")  
print(clientes_df.head(10).to_string(index=False))  
print(f"\nDistribuição de  
segmentos:\n{clientes_df['segmento'].value_counts()}")  
return clientes_df
```

```
clientes = segmentar_clientes(df)  
clientes.head()
```

RF08 – Calcular Estatísticas com NumPy

O projeto deverá usar diretamente o NumPy para calcular estatísticas dos dados. Deve-se demonstrar ao menos:

- Conversão de uma coluna do DataFrame para array NumPy;
- Uso de operações vetorizadas (sem loops);
- Uso de broadcasting ou operações entre arrays;
- Pelo menos 3 funções NumPy (`mean`, `std`, `median`, `percentile`, `sum`, etc.).

Exemplo:

```
def calcular_estatisticas_numpy(df):  
    """  
    Usa NumPy diretamente sobre arrays para calcular estatísticas de receita.  
    Demonstra três conceitos:  
    1. Operações vetorizadas  
    2. Broadcasting (operação escalar aplicada a todo o array de uma vez)  
    3. Boolean indexing (filtrar array com máscara booleana)  
    """  
  
    # .to_numpy() converte a Series do Pandas em um array NumPy puro.  
    # Isso é necessário para usar funções do NumPy diretamente e demonstrar  
    # que sabemos trabalhar com arrays além do DataFrame.  
    receitas = df["receita_total"].to_numpy()  
  
    # --- Estatísticas descritivas (operações vetorizadas) ---  
    # Cada função abaixo opera sobre o array inteiro  
    # "float()" garante que os valores sejam serializáveis em JSON (RF11)  
    stats = {  
        "media": float(np.mean(receitas)),  
        "mediana": float(np.median(receitas)),  
        "desvio_padrao": float(np.std(receitas)),  
        "total": float(np.sum(receitas)),  
        "p25": float(np.percentile(receitas, 25)),  
        "p75": float(np.percentile(receitas, 75)),  
    }  
  
    # --- Broadcasting: participação percentual de cada venda ---  
    # receitas.sum() é um escalar; NumPy o aplica a cada elemento do array,  
    # calculando o % de cada venda no total.
```

```
# Isso é broadcasting: operação entre um array e um escalar.
receitas_pct = (receitas / receitas.sum()) * 100
print(f" Participação das 5 maiores vendas no total:
"f"{np.sort(receitas_pct)[-5:].round(2)}%")

# --- Boolean indexing (filtro vetorizado) ---
# (receitas > stats["media"]) gera um array de True/False para cada linha;
# .sum() conta os True. Equivale a um for+if, mas muito mais eficiente.
acima_da_media = int((receitas > stats["media"]).sum())
stats["acima_da_media"] = acima_da_media

# Exibição – formato separado para inteiro evitar "12.00"
print("\n=== ESTATÍSTICAS COM NUMPY ===")
for k, v in stats.items():
    if k == "acima_da_media":
        print(f" {k}: {v} vendas")
    else:
        print(f" {k}: R$ {v:.2f}")

return stats

stats = calcular_estatisticas_numpy(df)
```

RF09 – Criar Visualizações com Matplotlib e Seaborn

O projeto deverá gerar ao menos **3 gráficos distintos**, exportados em arquivos PNG. Os gráficos devem ser informativos, com título, rótulos de eixos e legenda, quando aplicável.

Gráficos obrigatórios:

1. **Gráfico de linha:** Receita total por mês ao longo do tempo;
2. **Gráfico de barras:** Top 5 produtos ou categorias por receita;
3. **Gráfico adicional** (à sua escolha): dispersão, histograma, boxplot, mapa de calor (heatmap), pizza ou barras agrupadas.

Exemplo:

```
import matplotlib.pyplot as plt
import seaborn as sns
import os
```

```
def gerar_visualizacoes(df, metricas, output_dir="outputs/graficos"):  
    """  
    Gera e exporta 3 gráficos informativos em PNG:  
    1. Linha – receita total por mês (tendência ao longo do tempo)  
    2. Barras – top 5 produtos por receita (ranking)  
    3. Boxplot – distribuição de receita por região (dispersão e outliers)  
  
    sns.set_theme() aplica um estilo visual global a todos os gráficos  
    gerados na sessão – só precisa ser chamado uma vez.  
    """  
  
    os.makedirs(output_dir, exist_ok=True)  
    sns.set_theme(style="whitegrid", palette="muted")  
  
    meses_abrev = ["Jan", "Fev", "Mar", "Abr", "Mai", "Jun",  
                   "Jul", "Ago", "Set", "Out", "Nov", "Dez"]  
  
    # Gráfico 1: Linha – Receita por Mês  
    # Ideal para mostrar tendência e sazonalidade ao longo do tempo.  
    fig, ax = plt.subplots(figsize=(10, 5))  
    pm = metricas["por_mes"]  
    ax.plot(pm["mes"], pm["receita_total"], marker="o", linewidth=2)  
    ax.set_title("Receita Total por Mês")  
    ax.set_xlabel("Mês")  
    ax.set_ylabel("Receita (R$)")  
    ax.set_xticks(range(1, 13))  
    ax.set_xticklabels(meses_abrev, rotation=45)  
    fig.tight_layout()  
    fig.savefig(f"{output_dir}/receita_por_mes.png", dpi=120)  
    plt.show()  
    plt.close() # libera memória da figura anterior antes de criar a próxima  
  
    # Gráfico 2: Barras Horizontais – Top 5 Produtos  
    # Barras horizontais facilitam a leitura de rótulos longos (nomes de  
    produtos).  
    fig, ax = plt.subplots(figsize=(10, 5))  
    sns.barplot(data=metricas["top_produtos"], y="produto", x="receita_total",  
    ax=ax)
```

```
ax.set_title("Top 5 Produtos por Receita Total")
ax.set_xlabel("Receita Total (R$)")
ax.set_ylabel("Produto")
fig.tight_layout()
fig.savefig(f"{output_dir}/top_produtos.png", dpi=120)
plt.show()
plt.close()
```

```
# Gráfico 3: Boxplot – Distribuição de Receita por Região
# O boxplot mostra mediana, quartis e outliers de cada grupo,
# permitindo comparar não só o volume mas a variação interna
# de cada região – informação que um gráfico de barras não entrega.
fig, ax = plt.subplots(figsize=(10, 5))
sns.boxplot(data=df, x="regiao", y="receita_total", ax=ax)
ax.set_title("Distribuição de Receita por Região")
ax.set_xlabel("Região")
ax.set_ylabel("Receita por Venda (R$)")
plt.xticks(rotation=30)
fig.tight_layout()
fig.savefig(f"{output_dir}/dist_regiao.png", dpi=120)
plt.show()
plt.close()
print(f"3 gráficos salvos em: {output_dir}")
```

```
gerar_visualizacoes(df, metricas)
```

RF10 – Organizar o Código em Funções Reutilizáveis

Todo o fluxo de análise deve estar encapsulado em funções, e não em código solto espalhado pelas células. Espera-se que o estudante:

- Crie funções com parâmetros e retorno para cada etapa (carregar, limpar, transformar, agregar, visualizar);
- Documente cada função com uma docstring explicando o que ela faz;
- Reutilize ao menos uma função em mais de um contexto (ex.: a mesma limpeza usada para gerar a v1 e a v2);
- Use ao menos uma função lambda (por exemplo, na segmentação de clientes).

Opcional (bônus): criar uma função que recebe outra função como parâmetro (callback), demonstrando funções de ordem superior. Veja exemplos abaixo:

Exemplos de uso:

```
# FUNÇÕES DE ORDEM SUPERIOR
```

```
# Uma função de ordem superior é aquela que recebe outra função
```

```
# como argumento (callback) ou retorna uma função.
```

```
# Isso permite generalizar comportamentos: em vez de criar uma função
```

```
# para cada transformação possível, criamos uma que aceita qualquer
```

```
# função como parâmetro.
```

```
def aplicar_transformacao(df, coluna, funcao):
```

```
    """
```

```
    Função de ordem superior: aplica qualquer função (incluindo lambdas)
    a uma coluna do DataFrame, criando uma coluna '_transformado'.
```

```
    Parâmetros:
```

```
        df      : DataFrame de entrada
```

```
        coluna  : nome da coluna a transformar
```

```
        funcao  : função (ou lambda) a aplicar – o 'callback'
```

```
    Retorna uma cópia do DataFrame com a nova coluna; não modifica o original.
```

```
    """
```

```
    df = df.copy()
```

```
    df[f"{coluna}_transformado"] = df[coluna].apply(funcao)
```

```
    return df
```

```
# --- Uso 1: classificar vendas por ticket com lambda ---
```

```
# lambda como callback – 2º uso distinto de lambda no projeto (1º foi em RF07)
```

```
df_demo = aplicar_transformacao(
```

```
    df,
```

```
    "receita_total",
```

```
    lambda x: "Alto" if x > 5000 else "Normal"
```

```
)
```

```
print("=== EXEMPLO: classificação por ticket ===")
```

```
print(df_demo[["receita_total", "receita_total_transformado"]].head())
```

```
# --- Uso 2: arredondar receita em milhares com lambda ---
```

```
# Demonstra que a mesma função aceita qualquer transformação
```

```
df_demo2 = aplicar_transformacao(
    df,
    "receita_total",
    lambda x: round(x / 1000, 2)
)
print("\n=== EXEMPLO: receita em milhares (R$ k) ===")
print(df_demo2[["receita_total", "receita_total_transformado"]].head())
# Nota: df_demo e df_demo2 são apenas demonstrações – df principal não é
alterado.
```

RFII – Ler e Escrever Arquivos (CSV e JSON)

O projeto deverá demonstrar a leitura e a escrita de arquivos em pelo menos dois formatos.

```
import json

def exportar_resultados(metricas, clientes, stats):
    """
    Exporta os resultados da análise em dois formatos:
    - CSV : métricas mensais e segmentação de clientes
    - JSON: estatísticas gerais calculadas com NumPy

    Após exportar o JSON, faz a leitura de volta com json.load()
    para confirmar que o arquivo foi gravado corretamente –
    demonstrando leitura e escrita de JSON no mesmo fluxo.
    """
    os.makedirs("outputs", exist_ok=True)

    # --- Exportação CSV ---
    # encoding="utf-8-sig" adiciona um BOM (Byte Order Mark) ao arquivo.
    # Isso garante que o Excel abra o CSV com acentos corretamente;
    metricas["por_mes"].to_csv(
        "outputs/metricas_por_mes.csv", index=False, encoding="utf-8-sig"
    )
    print("CSV exportado: outputs/metricas_por_mes.csv")

    clientes.to_csv(
        "outputs/segmentacao_clientes.csv", index=False, encoding="utf-8-sig"
    )
```

```
print("CSV exportado: outputs/segmentacao_clientes.csv")

# --- Exportação JSON ---
# ensure_ascii=False permite gravar acentos como caracteres reais (ã, é)
# round(float(v), 2) converte para float antes de arredondar,
# evitando comportamento inesperado com o campo 'acima_da_media' (int).
stats_serializaveis = {k: round(float(v), 2) for k, v in stats.items()}
caminho_json = "outputs/estatisticas_gerais.json"
with open(caminho_json, "w", encoding="utf-8") as f:
    json.dump(stats_serializaveis, f, indent=2, ensure_ascii=False)
    # indent=2 formata o JSON com recuo de 2 espaços, mais legível que uma
linha só
print(f"JSON exportado: {caminho_json}")

# --- Leitura de volta para confirmar ---
with open(caminho_json, encoding="utf-8") as f:
    lido = json.load(f)
print("\nJSON lido de volta para confirmação:")
print(json.dumps(lido, indent=2, ensure_ascii=False))

exportar_resultados(metricas, clientes, stats)
```

RF12 – Consolidar a Análise e Salvar o Dataset Final

Ao final, o notebook deve consolidar a análise completa — do dado bruto ao dataset final salvo. Como as funções foram chamadas progressivamente em cada RF anterior, esta célula fecha o ciclo: salva o dataset enriquecido em data/final/, e verifica se todos os arquivos esperados foram gerados.

A escolha da versão base é do aluno: v1 mantém os outliers para análise completa; v2 os remove. Documente sua escolha no arquivo README.md.

Exemplo de uso:

```
# RF12 – CONSOLIDAÇÃO FINAL
# Neste ponto do notebook, todas as etapas já foram executadas:
# Esta célula salva o dataset final em data/final/

# --- Decisão de versão ---
```

Escolhemos `df_v2` (outliers removidos) como base da análise final. O aluno pode optar pela `v1` se quiser preservar todos os registros – ambas as versões estão salvas em “`data/processed/`” para consulta futura.

```
os.makedirs("data/final", exist_ok=True)
df.to_csv("data/final/vendas_final.csv", index=False)
print("Dataset final salvo em: data/final/vendas_final.csv")
```

4.3. REQUISITOS TÉCNICOS

Conteúdo	Obrigatório?	Como Demonstrar
Google Colab ou VS Code	Sim	Usar como ambiente de desenvolvimento
Extensões/ferramentas utilizadas	Sim	Citar no README
Versionamento	Sim	Commits no GitHub
GitHub	Sim	Repositório público
Lógica de programação	Sim	Cálculos, condicionais e repetições
Tipos de dados	Sim	int, float, str, bool, list, dict
if / elif / else	Sim	Classificação de segmentos e faixas
Operadores aritméticos, lógicos e relacionais	Sim	Cálculo de receita, filtros, comparações
for / while	Sim	Loops de exibição ou iteração de dados
Funções com parâmetros e retorno	Sim	Cada etapa do notebook deve ser encapsulada em uma função com parâmetros e <code>return</code> explícito. Exemplos: <code>limpar_dados(df)</code> retorna <code>(df_limpo, relatorio)</code> ; <code>calcular_metricas(df)</code> retorna o dicionário <code>metricas</code>
Funções lambda	Sim	Pelo menos 2 usos distintos
Função que recebe função como argumento	Opcional	Crie uma função com um parâmetro <code>funcao</code> e chame-a passando uma lambda como argumento. Exemplo: <code>aplicar_transformacao(df, "receita_total", lambda x: ...)</code>
Listas, dicionários e estruturas compostas	Sim	Dataset, métricas, relatório de limpeza

Leitura de CSV	Sim	<code>pd.read_csv()</code>
Escrita de CSV	Sim	<code>.to_csv()</code>
Módulo datetime	Sim	Extração de mês, trimestre, ano
Expressões regulares (re)	Sim	Limpeza de strings com <code>re.sub()</code> ou <code>re.compile()</code> Exemplo: <code>re.sub(r"\s+", " ", texto)</code> colapsa múltiplos espaços em um
Pandas – Series e DataFrames	Sim	Estrutura principal de dados
Pandas – Filtros e seleções	Sim	Filtros com condicionais booleanas
Pandas – groupby	Sim	Agregações por mês, produto, região
Pandas – merge ou pivot	Opcional	Pode ser usado para enriquecer dados
NumPy – Arrays	Sim	Conversão de colunas e operações
NumPy – Operações vetorizadas	Sim	Sem loops em cálculos numéricos
NumPy – Broadcasting	Sim	Aplique uma operação escalar diretamente sobre um array sem usar loop. Exemplo: <code>receitas_pct = (receitas / receitas.sum()) * 100</code> , o escalar <code>receitas.sum()</code> é aplicado a cada elemento do array automaticamente
NumPy – np.select	Sim	Transformação condicional vetorizada
Transformações condicionais	Sim	Classificação de faixas de receita
Matplotlib – gráfico de linha	Sim	Receita por mês
Matplotlib/Seaborn – gráfico de barras	Sim	Top produtos
Matplotlib/Seaborn – gráfico adicional	Sim	Boxplot, histograma, pizza ou dispersão
Exportação de gráficos (.png)	Sim	<code>plt.savefig()</code>
Módulos e importação	Sim	<code>import pandas, import numpy, etc.</code>

4.4. VERSIONAMENTO COM GITHUB

O projeto deverá ter um repositório público no GitHub, organizado segundo as boas práticas de um projeto de análise e exploração de dados, refletindo a estrutura de pastas descrita no item 4.1 (data/raw, data/processed com v1 e v2, e data/final).

O aluno deverá trabalhar registrando o progresso da análise por meio de *commits* bem descritos.

4.5. GRAVAÇÃO DE VÍDEO

Além do desenvolvimento das tarefas, você deverá gravar um vídeo, com tempo máximo de 3 minutos, abordando os seguintes tópicos:

- Qual o objetivo do sistema? Demonstre o notebook rodando do início ao fim.
- O que é necessário instalar ou configurar para executar o projeto?
- O que poderia ser melhorado no seu código?

Você poderá gravar na vertical ou na horizontal. É importante que seu rosto apareça, que esteja em um local bem iluminado e que sua voz seja ouvida com clareza. Para a entrega do vídeo, coloque-o em uma pasta no Google Drive com permissão de leitura para qualquer pessoa com o link (ou publique no YouTube como vídeo não listado) e compartilhe o link na submissão no AVA. Uma dica valiosa é inserir o link do vídeo diretamente no README.md do repositório no GitHub. É obrigatório que qualquer pessoa consiga acessar o vídeo pelo link fornecido.

5. CRITÉRIOS DE AVALIAÇÃO

A tabela abaixo apresenta os critérios que serão avaliados durante a correção do projeto.

O mesmo possui variação de nota de 0 (zero) a 10 (dez) e peso de 25% na avaliação do Módulo 01.

Serão desconsiderados e lhes será atribuída a nota 0 (zero) os projetos que apresentarem plágio de soluções encontradas na internet ou de outros colegas.

Lembre-se: Você está livre para consultar materiais, documentação, exemplos e ferramentas de apoio, desde que consiga adaptar a solução ao desafio proposto e explicar o código entregue com segurança e confiança.

Apresentação do Projeto (4,0)				
Nº	Critério de Avaliação	Zero	Parcial	Máximo
1	Estruturou o README.md contendo obrigatoriamente: objetivo do projeto, instruções de instalação e execução, lista de conceitos de Python e de análise de dados aplicados, e links para o repositório ou vídeo.	0	0,75	1,0

2	Entregou o repositório público no GitHub com a estrutura de pastas do projeto (data/raw, data/processed com v1 e v2, e data/final) e um histórico de commits descritivos que reflete a evolução lógica da análise.	0	0,75	1,0
3	Organizou o fluxo de desenvolvimento, mantendo a correlação direta entre as tarefas listadas e o código implementado.	0	0,75	1,0
4	Demonstrou fluidez técnica ao explicar, por meio de um vídeo, o funcionamento da análise, as decisões de implementação e as oportunidades de melhoria identificadas no projeto.	0	0,75	1,0

Desenvolvimento do Projeto (6,0)

Nº	Critério de Avaliação	Zero	Parcial	Máximo
5	O projeto carregou, inspecionou e limpou corretamente o dataset, tratando valores nulos, datas inválidas e strings sujas, utilizando datetime e expressões regulares.	0	0,50	1,00
6	Criou colunas derivadas, calculou métricas agregadas com groupby, realizou segmentação de clientes com lambda e transformações condicionais com np.select.	0	0,75	1,50
7	Usou NumPy para operações vetorizadas, broadcasting e cálculo de estatísticas, demonstrando processamento sem loops sobre arrays.	0	0,40	0,75
8	Gerou ao menos 3 gráficos distintos e informativos com Matplotlib e/ou Seaborn, exportados em PNG, com títulos, rótulos e boa apresentação visual.	0	0,50	1,00
9	Detectou e tratou outliers em colunas numéricas (IQR ou z-score) e gerou as duas versões dos dados (v1 com outliers e v2 tratada), ambas salvas nas pastas corretas em data/processed/.	0	0,60	1,25
10	Organizou o código em funções reutilizáveis, usou uma função lambda e leu e escreveu arquivos CSV e JSON corretamente.	0	0,25	0,50

6. CHECKLIST FINAL DE ENTREGA

Antes de enviar no AVA, confira:

[] Criei o repositório público no GitHub

- ☐ Criei o notebook dataview.ipynb
- ☐ Criei o README.md
- ☐ Gerei ou incluí o dataset vendas.csv
- ☐ Inspecionei os dados com `.shape`, `.dtypes`, `.isnull()`
- ☐ Limpei os dados (nulos, datas inválidas, strings sujas)
- ☐ Detectei outliers (IQR ou z-score) em colunas numéricas
- ☐ Gerei a versão v1 (com outliers) e a versão v2 (outliers tratados)
- ☐ Salvei v1 e v2 nas pastas corretas em `data/processed/`
- ☐ Usei o módulo `datetime` para extrair mês, trimestre e ano
- ☐ Usei expressões regulares (`re`) para limpeza de strings
- ☐ Criei colunas derivadas (`receita_total`, `trimestre`, `faixa_receita_item`)
- ☐ Usei `groupby` para agregar métricas por mês, produto e região
- ☐ Usei `np.select` ou equivalente para transformação condicional vetorizada
- ☐ Usei NumPy para estatísticas (`mean`, `std`, `median`, `percentile`)
- ☐ Demonstrei broadcasting ou operações vetorizadas com NumPy
- ☐ Usei funções `lambda`
- ☐ Reutilizei ao menos uma função em mais de um contexto (ex.: limpeza p/ v1 e v2)
- ☐ Exportei resultados em CSV com `to_csv()`
- ☐ Exportei os resultados em JSON com `json.dump()`
- ☐ Li o JSON exportado com `json.load()` para confirmar
- ☐ Gerei gráfico de linha (receita por mês)
- ☐ Gerei gráfico de barras (top produtos ou categorias)
- ☐ Gerei um terceiro gráfico (boxplot, histograma, pizza ou dispersão)
- ☐ Exportei os gráficos em PNG com `plt.savefig()`
- ☐ Salvei o dataset final em `"data/final/vendas_final.csv"`
- ☐ Fiz commits descritivos no GitHub
- ☐ Gravei o vídeo de até 3 minutos

[] Coloquei o vídeo no Google Drive com permissão correta (ou YouTube não listado)

[] Enviei os links no AVA antes do prazo de entrega

7. MODELO DE README.md (SUGESTÃO)

MiniProjeto DataView

Sobre o projeto

O DataView PY é um projeto de análise exploratória de dados (EDA) de vendas, desenvolvido em Python em um notebook. O notebook lê, limpa, transforma, analisa e visualiza um dataset de vendas, gerando métricas e insights.

O que o projeto analisa

- Receita total e volume de vendas por mês e trimestre
- Top produtos e categorias por receita
- Desempenho por região
- Segmentação de clientes por nível de gasto (Bronze, Prata, Ouro)
- Comparação entre os dados com e sem tratamento de outliers (v1 e v2)
- Exportação de relatórios em CSV e JSON

Objetivo

Praticar os principais conceitos:

- Lógica de programação com Python
- Variáveis, tipos de dados e operadores
- Condicionais (if, elif, else) e repetição (for, while)
- Funções com parâmetros, retorno e funções lambda
- Funções reutilizáveis
- Leitura e escrita de arquivos CSV e JSON
- Módulo datetime para manipulação de datas
- Expressões regulares com o módulo re
- Pandas: DataFrames, limpeza, groupby, filtros e transformações
- NumPy: arrays, operações vetorizadas, broadcasting
- Detecção e tratamento de outliers (IQR ou z-score)
- Matplotlib e Seaborn: gráficos, customização e exportação em PNG
- Uso básico do GitHub

Como executar

No Google Colab (recomendado)

1. Faça upload do notebook dataview.ipynb
2. Abra o arquivo carregado no Colab
3. Execute as células na ordem, de cima para baixo

Localmente com VS Code

1. Instale o Python 3.10+ e o VS Code.
2. Instale as dependências:
`pip install pandas numpy matplotlib seaborn`

Estrutura do projeto

```
projeto/  
|-- data/  
|   |-- raw/                # Dataset bruto gerado/baixado  
|   |-- processed/  
|       |-- v1_com_outliers/ # Dados de limpeza geral, outliers mantidos  
|       |-- v2_outliers_tratado/ # Limpeza v1 + tratamento de outliers  
|   |-- final/              # Dataset escolhido para uso futuro  
|-- notebooks/  
|   |-- dataview.ipynb      # Notebook principal de EDA  
|-- outputs/  
|   |-- metricas_por_mes.csv  
|   |-- segmentacao_clientes.csv  
|   |-- estatisticas_gerais.json  
|   |-- graficos/  
|-- README.md
```

Ferramentas utilizadas

- Python 3.10+
- Google Colab / VS Code
- Bibliotecas: pandas, numpy, matplotlib, seaborn, re, datetime, os, random
- GitHub para versionamento

Vídeo de demonstração

[Inserir link do Google Drive ou YouTube aqui]
